

GIT

Crear el `readme.txt`:

```
echo "Este repositorio contendra las actividades y practicas realizadas en la unidad." >
readme.txt
```

Crear el `.gitignore`:

```
echo "*.log" > .gitignore
```

¿Qué hace?: Crea un archivo de texto con el contenido que pones entre comillas.

¿Para qué sirve?:

- `readme.txt`: Es la carta de presentación de tu proyecto donde explicas de qué trata.
- `.gitignore`: Es la lista de "archivos invisibles". Al escribir `*.log` dentro, le estás diciendo a Git: *"Ignora cualquier archivo que termine en .log (archivos de registro del sistema) y nunca lo guardes en el historial"*.

Paso 2: Añadir los archivos a Git

Prepara los archivos para guardarlos:

```
git add readme.txt .gitignore
```

¿Qué hace?: Coloca los archivos en la **mesa de preparación** (en Git se llama *Staging Area*).

¿Para qué sirve?: Git no guarda los cambios automáticamente. Primero tienes que poner en la "mesa" los archivos exactos que quieres incluir en la siguiente foto.

Paso 3: Guardar los archivos (Primer *commit*)

Guarda los cambios con este comando:

```
git commit -m "Primer commit: Creacion de readme.txt y .gitignore"
```

¿**Qué hace?**: Toma la "foto" (guardado definitivo) de todo lo que tenías puesto en la mesa de preparación.

¿**Para qué sirve?**: Es el comando que realmente guarda una versión en el historial. El texto después de `-m` es un mensaje que dejas escrito para recordar qué hiciste en esa versión.

Paso 4: Asegurar que estás en la rama `master`

Ahora que ya existe un `commit`, la rama por defecto de Ubuntu suele llamarse `master`.

Para estar 100% seguro de que se llama `master` (como pide tu profesora), vuelve a probar ahora el comando que dio fallo:

```
git branch -M master
```

(Esta vez lo ejecutará sin ningún error).

¿**Qué hace?**: Renombra la línea de tiempo principal (rama) de tu proyecto como `master`.

¿**Para qué sirve?**: Git moderno suele llamar a la rama principal `main`, pero como tu profesora te pidió explícitamente que se llame `master`, este comando le cambia el nombre.

Paso 5: Comprobar el resultado

Verifica que todo está perfecto ejecutando:

```
git status
```

Te dirá que estás en la rama `master` (`On branch master`) y que todo está limpio.

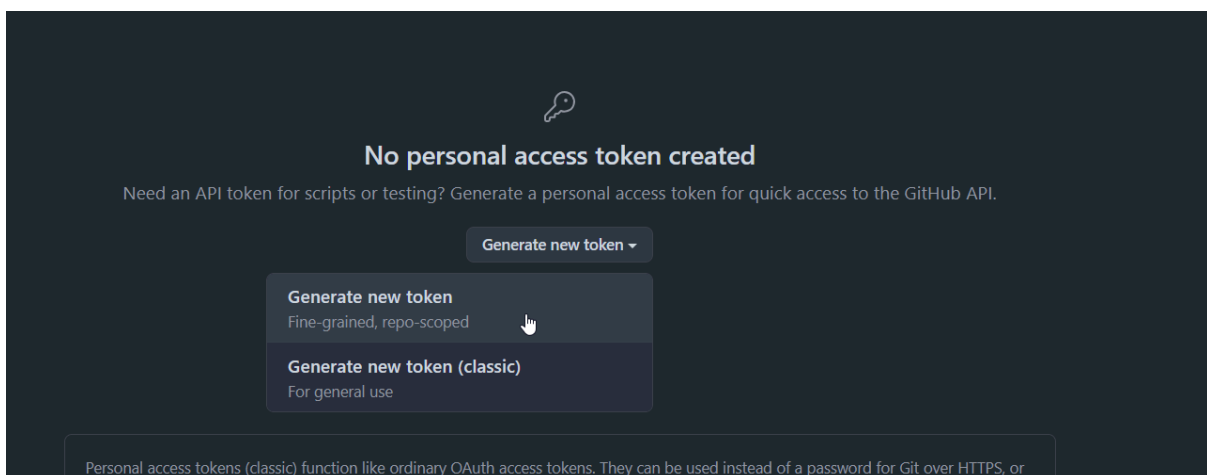
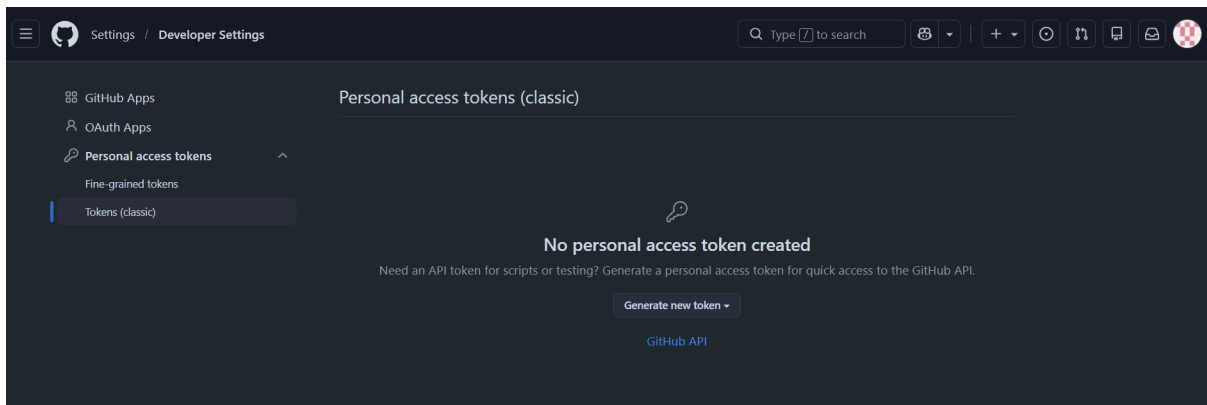
¿**Qué hace?**: Te muestra la situación actual del proyecto.

¿**Para qué sirve?**: Te dice si tienes archivos nuevos sin guardar, si hay cambios pendientes o si todo está al día y limpio.

SEGUNDA ACTIVIDAD

Configurar un Token de Acceso Personal (PAT) en GitHub

1. Entra a [GitHub](#) e inicia sesión.
2. Ve a Settings > Developer Settings > Credentials > Personal Access Tokens > Tokens (classic).
3. Genera un nuevo token y asígnale una descripción (ej. Token Prácticas), selecciona el alcance repo y genera el token.
4. Copia y guarda el token para usarlo como contraseña cuando Git te pida credenciales.



GitHub Apps

OAuth Apps

Personal access tokens

Fine-grained tokens

Tokens (classic)

New fine-grained personal access token

Create a fine-grained, repository-scoped token suitable for personal API use and for using Git over HTTPS.

Token name *

Token_Practicas

✓ "Token_Practicas" is available.

A unique name for this token. May be visible to resource owners or users with possession of the token.

Description

De prueba en clases

Resource owner

Bolenita

The token will only be able to make changes to resources owned by the selected resource owner. Tokens can always read all public repositories.

Expiration

30 days (Nov 04, 2026)

The token will expire on the selected date

New personal access token

Your new personal access token **Token_Practicas** will be ready for use immediately. It will expire on **Wednesday, November 04, 2026**.

Cancel Generate token

GitHub Apps

OAuth Apps

Personal access tokens

Fine-grained tokens

Tokens (classic)

Fine-grained personal access tokens

Generate new token

These are fine-grained, repository-scoped tokens suitable for personal [API](#) use and for using Git over HTTPS.

Token_Practicas

Never used • Expires on Wed, Nov 4 2026

Delete

Make sure to copy your personal access token now as you will not be able to see this again.

github_pat_11BM443IY0KT8NfL6jyhaf_D2FN6SedDX7t6vSskbb71aRwI1wQ1GLh4NXXA5YaI7L0SE4P3mEX0

Crear el repositorio remoto y conectar con local

1. Crea un nuevo repositorio GitHub llamado **UD01_Actividades** (déjalo vacío: sin README, sin .gitignore ni Licencia).

Fine-grained personal access tokens

These are fine-grained, repository-scoped tokens suitable for personal [API](#) use and for using Git

Type to search

New issue

New repository

Import repository

Owner * / Repository name *

Bolenita / UD01_Actividades

✔ UD01_Actividades is available.

Great repository names are short and memorable. How about [fuzzy-invention?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility * Choose who can see and commit to this repository Public

Add README READMEs can be used as longer descriptions. [About READMEs](#) Off

Add .gitignore .gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

Add license Licenses explain how others can use your code. [About licenses](#) No license

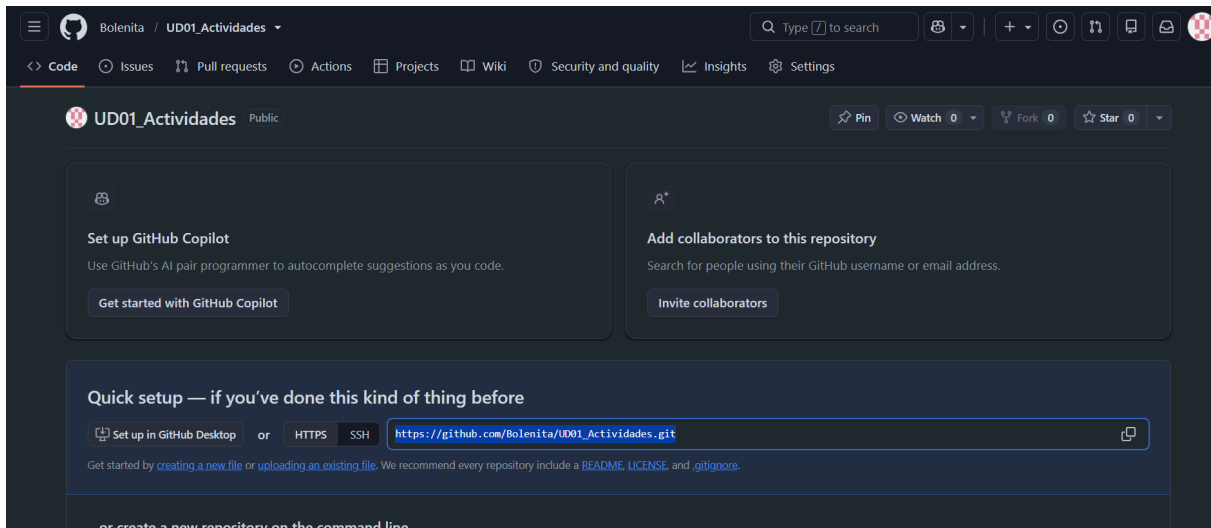
Create repository

2. En tu repositorio local renombra la rama master a main manteniendo el historial y comprueba que la rama local se ha renombrado a main correctamente.

```
usuario@MV-Ubuntu:~/Escritorio/practicas_git$ git branch -m master main
usuario@MV-Ubuntu:~/Escritorio/practicas_git$
```

¿Qué hace este comando?: La opción **-m** (move/rename) le cambia el nombre a la rama actual sin borrar ningún archivo ni el historial de *commits*.

3. Asocia el repositorio remoto con tu proyecto local:



```
usuario@MV-Ubuntu:~/Escritorio/practicas_git$ git remote add origin https://github.com/Bolenita/UD01_Actividades.git
usuario@MV-Ubuntu:~/Escritorio/practicas_git$
```

¿Cómo verificar que funcionó?: Ejecuta `git remote -v`. Debe mostrarte dos líneas con la dirección de GitHub.

```
usuario@MV-Ubuntu:~/Escritorio/practicas_git$ git remote add origin https://github.com/Bolenita/UD01_Actividades.git
usuario@MV-Ubuntu:~/Escritorio/practicas_git$ git remote -v
origin https://github.com/Bolenita/UD01_Actividades.git (fetch)
origin https://github.com/Bolenita/UD01_Actividades.git (push)
usuario@MV-Ubuntu:~/Escritorio/practicas_git$
```

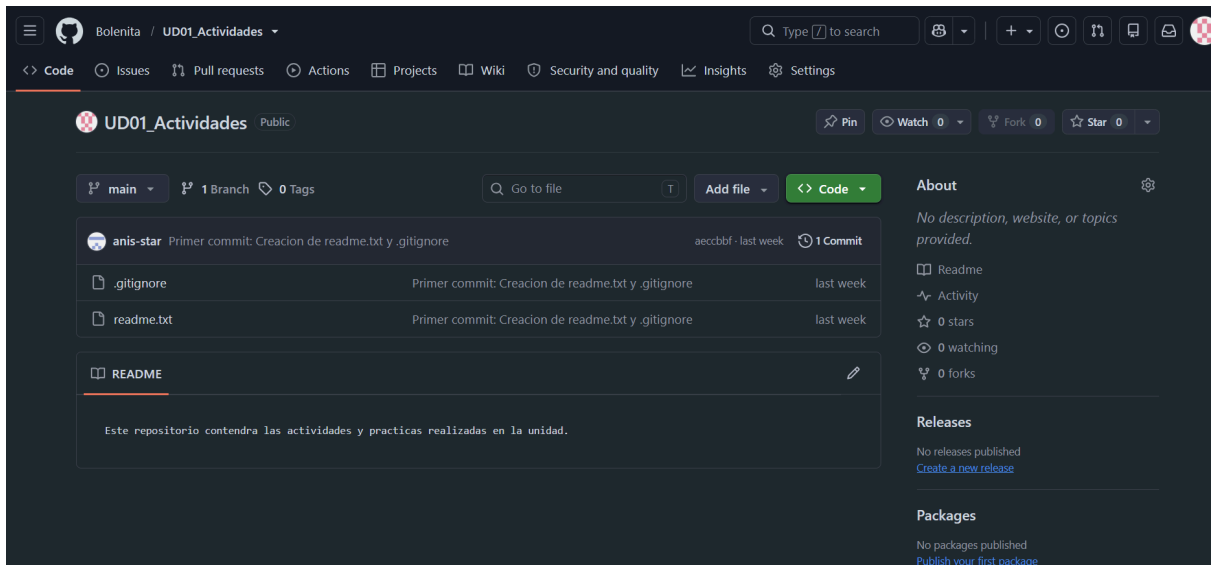
he tenido que volver a crear el token con repo marcado para que tenga permisos

Subir cambios y verificar en GitHub

1. Sube la rama main al repositorio remoto (Si te solicita credenciales, introduce tu usuario de GitHub y pega el Token como contraseña).
2. Entra en GitHub y comprueba que se han subido los archivos a la rama main.

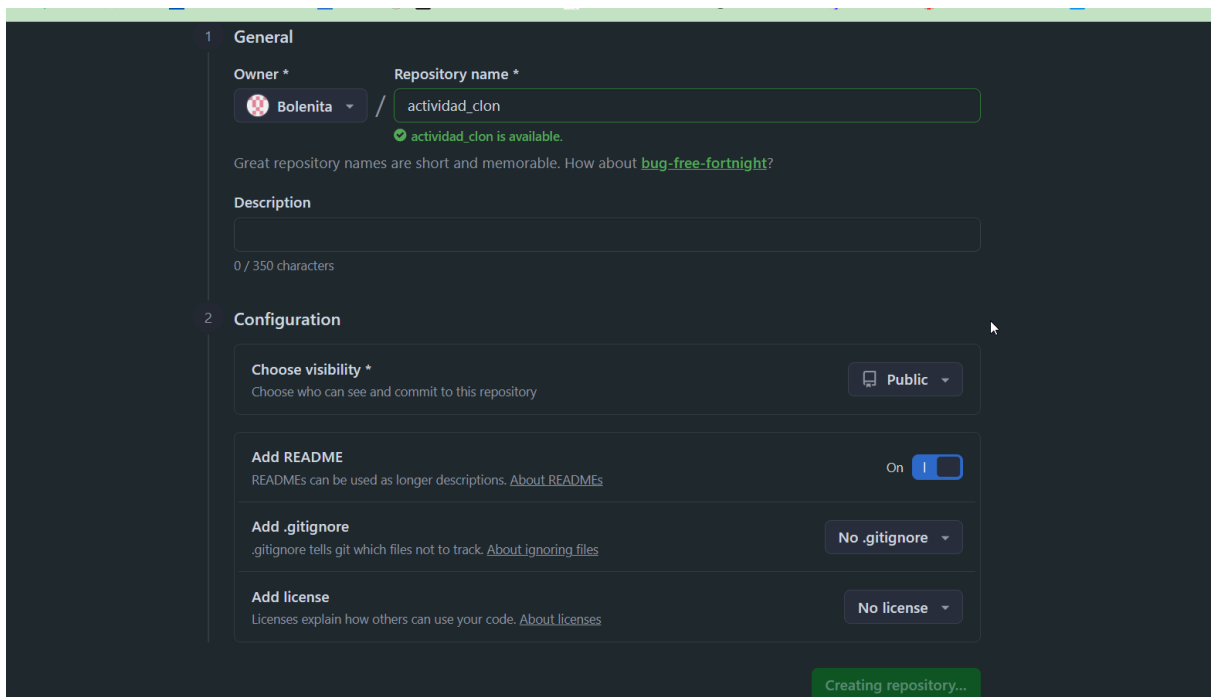
`git push -u origin main`

```
usuario@MV-Ubuntu:~/Escritorio/practicas_git$ git remote set-url origin https://ghp_WZu9knyCnc0JYosdBYI9pEPKRW8WY80uZzGb@github.com/Bolenita/UD01_Actividades.git
usuario@MV-Ubuntu:~/Escritorio/practicas_git$ git push -u origin main
Enumerando objetos: 100% (4/4), listo.
Comprimiendo delta usando hasta 2 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (4/4), 354 bytes | 354.00 KiB/s, listo.
Total 4 (delta 0), reusado 0 (delta 0)
To https://github.com/Bolenita/UD01_Actividades.git
* [new branch] main -> main
Rama 'main' configurada para hacer seguimiento a la rama remota 'main' de 'origin'.
usuario@MV-Ubuntu:~/Escritorio/practicas_git$
```



Sincronizar los cambios con el repositorio local

1. En tu terminal local, descarga la versión más reciente del repositorio remoto del proyecto **actividad_clon**
2. Comprueba cómo se ha actualizado la carpeta local y que el archivo prueba1.txt no está.
3. Compruebas tu historial local ¿Qué aparece registrado sobre el archivo prueba1.txt?



```

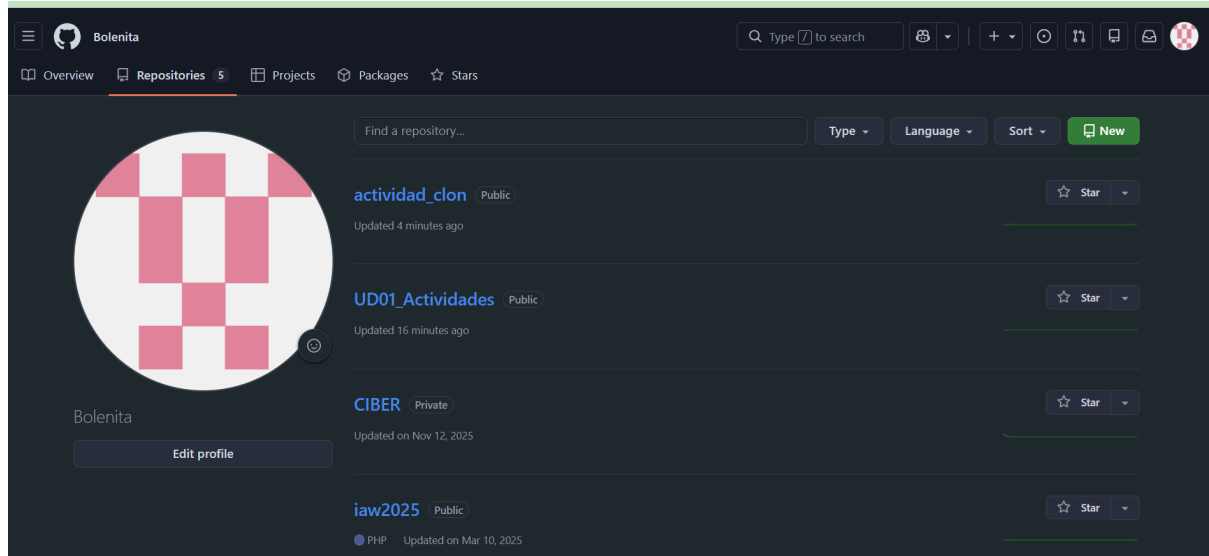
usuario@MV-Ubuntu:~/Escritorio$ git clone https://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@github.com/Bolenita/actividad_clon.git
Clonando en 'actividad_clon'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Desempaquetando objetos: 100% (3/3), 860 bytes | 860.00 KiB/s, listo.
usuario@MV-Ubuntu:~/Escritorio$

```

```

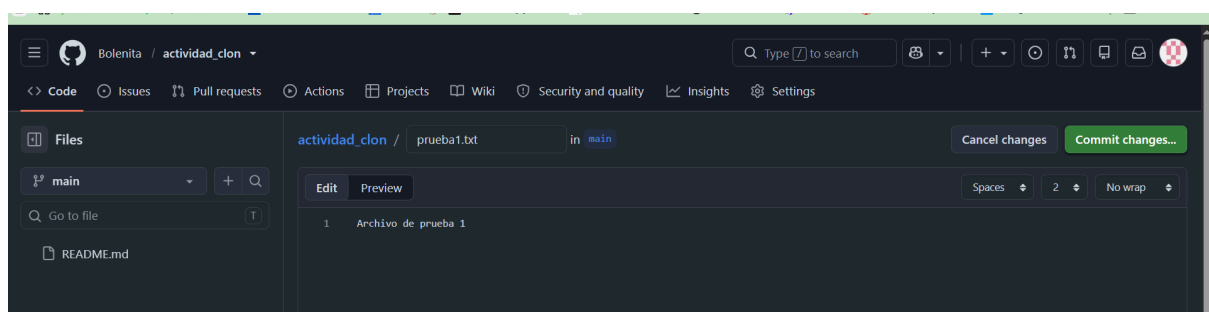
Desempaquetando objetos: 100% (3/3), 800 bytes | 800.00 KiB/s, listo.
usuario@MV-Ubuntu:~/Escritorio$ ls
actividad_clon practicas_git
usuario@MV-Ubuntu:~/Escritorio$ cd actividad_clon/
usuario@MV-Ubuntu:~/Escritorio/actividad_clon$ ls
README.md
usuario@MV-Ubuntu:~/Escritorio/actividad_clon$

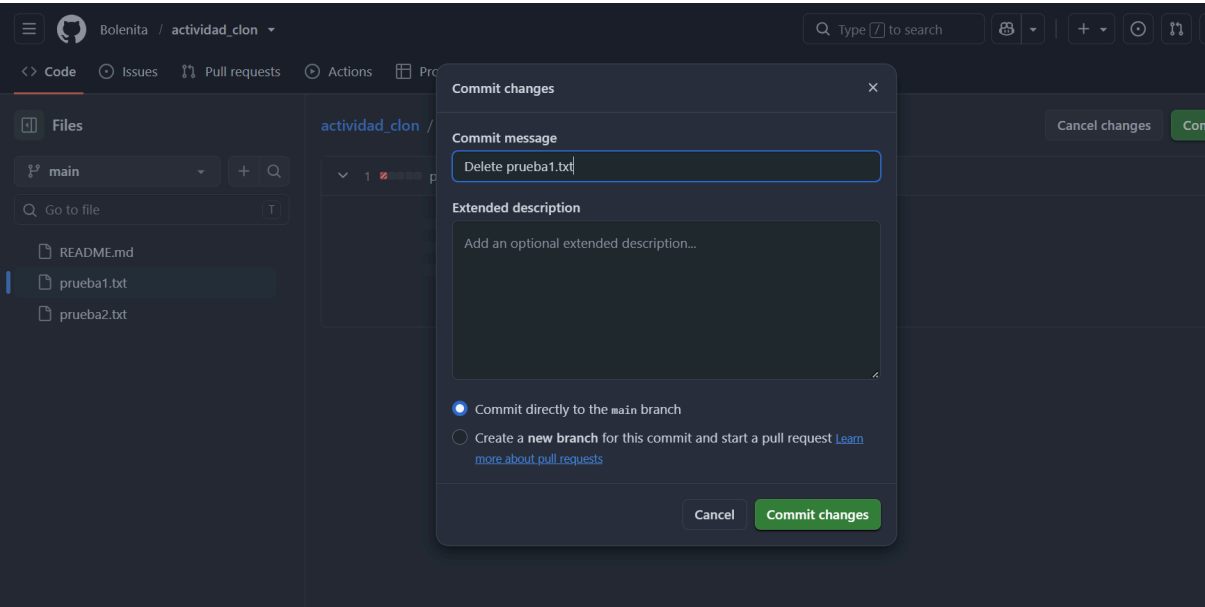
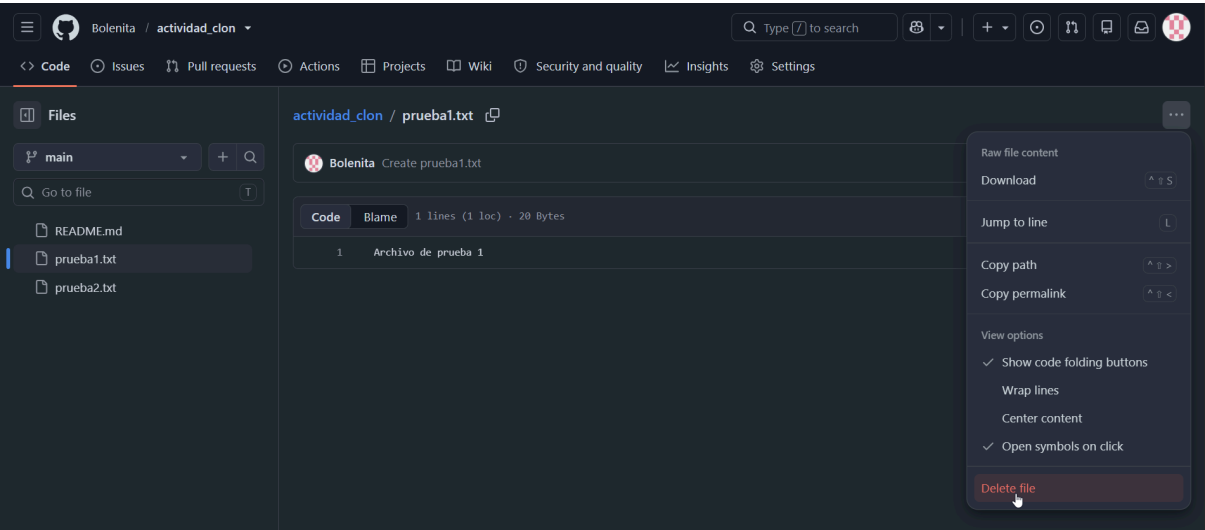
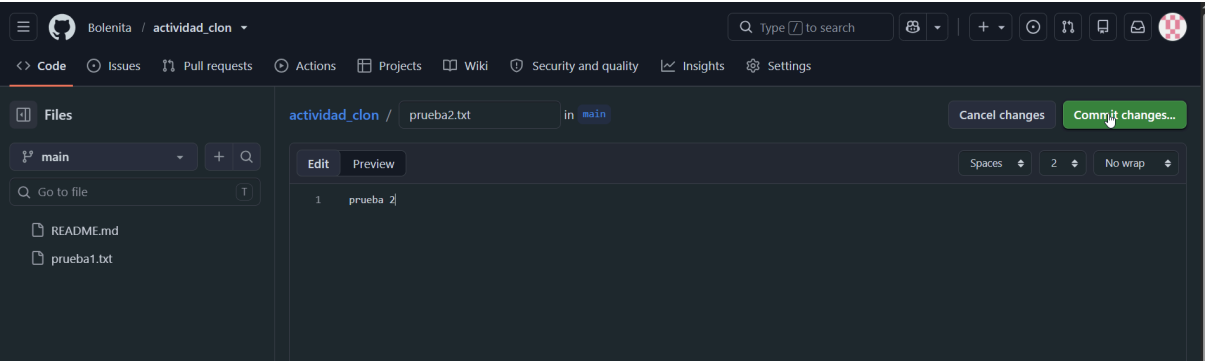
```

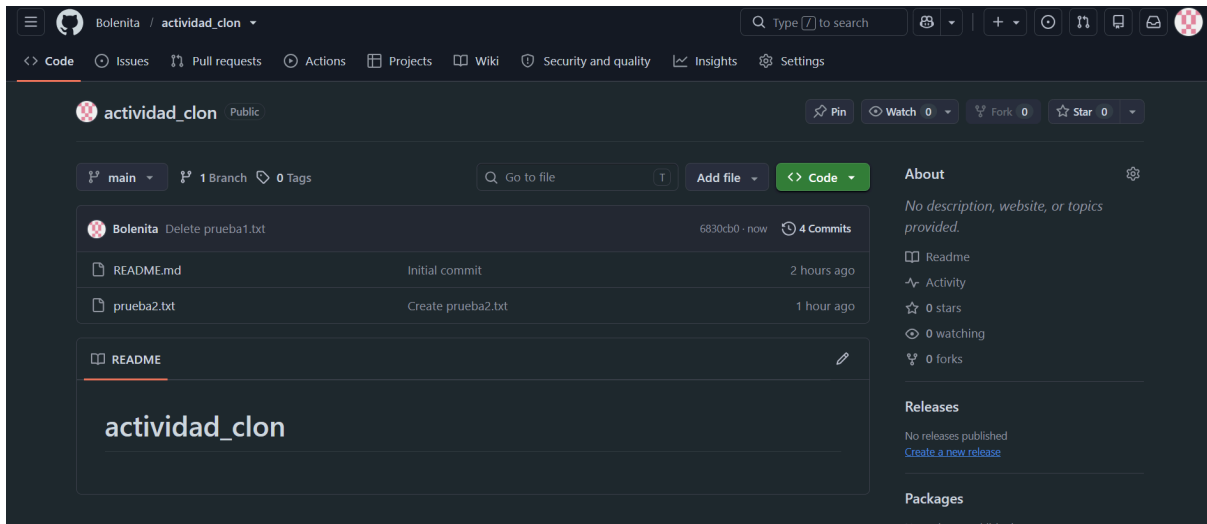


Eliminación de un fichero en remoto

1. En el proyecto **actividad_clon** en GitHub, crea dos archivos prueba1.txt y prueba2.txt y haz commit.
2. A continuación, elimina el archivo prueba1.txt de GitHub y confirma el commit.







Sincronizar los cambios con el repositorio local

1. En tu terminal local, descarga la versión más reciente del repositorio remoto del proyecto **actividad_clon**
2. Comprueba cómo se ha actualizado la carpeta local y que el archivo prueba1.txt no está.
3. Compruebas tu historial local ¿Qué aparece registrado sobre el archivo prueba1.txt?

```
usuario@MV-Ubuntu:~/Escritorio/actividad_clon$ git pull
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 8 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Desempaquetando objetos: 100% (8/8), 2.58 KiB | 2.58 MiB/s, listo.
Desde https://github.com/Bolenita/actividad_clon
  afefb00..6830cb0  main    -> origin/main
Actualizando afefb00..6830cb0
Fast-forward
 prueba2.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 prueba2.txt
usuario@MV-Ubuntu:~/Escritorio/actividad_clon$
```

¿Cómo verificar que funcionó?: Verás un resumen en la terminal donde dirá **prueba2.txt (mode 100644)** y **prueba1.txt (deleted)**.

```
usuario@MV-Ubuntu:~/Escritorio/actividad_clon$ ls
prueba2.txt  README.md
usuario@MV-Ubuntu:~/Escritorio/actividad_clon$
```

git log --oneline

```
prueba2.txt README.md
usuario@MV-Ubuntu:~/Escritorio/actividad_clon$ git log --oneline
6830cb0 (HEAD -> main, origin/main, origin/HEAD) Delete prueba1.txt
9b9af14 Create prueba2.txt
f027340 Create prueba1.txt
afeb00 Initial commit
usuario@MV-Ubuntu:~/Escritorio/actividad_clon$
```

Esto demuestra que Git no "olvida" las cosas borradas: guarda el registro completo de cuándo fue creado el archivo y cuándo fue eliminado.

Crear una rama de trabajo y colaborar

1. Vuelve al directorio del primer proyecto **UD01_Actividades**
2. Crea y cámbiate a una nueva rama con el nombre del *rama01_XXX* (Dónde XXX son tus iniciales)

git checkout -b rama01_AMOS

```
usuario@MV-Ubuntu:~/Escritorio/practicas_git$ git checkout -b rama01_AMOS
Cambiado a nueva rama 'rama01_AMOS'
usuario@MV-Ubuntu:~/Escritorio/practicas_git$
```

git checkout: Es el comando de Git para cambiarte de una rama a otra.
-b: Es la opción (*flag*) que le indica a Git: *"crea una rama nueva antes de cambiarte"*.
rama01_XXX: Es el **nombre exacto** que le estás dando a esa nueva rama.

3. Añade a esta rama el documento con las capturas que has ido haciendo durante la actividad.
4. Ve a Settings > Collaborators > Add people e invita a la profesora **pilarpr**